

Universidad Nacional Autónoma de México.



Facultad de Ingeniería.

Sistemas Distribuidos.

Profesor: Ing. Óscar Manuel Ruíz Hurtado.

Semestre 2027-1

Laboratorio S11.

Grupo: 2

Alumno:

Flores Herrera Oswaldo

Cd. Universitaria a 07 de noviembre de 2026

Creé las cuentas de Ana, Beto y Carla, hice una transferencia de 200 de Ana a Beto (respondió COMPLETADA con saldo de Ana tras el cargo de 638.00) y pedí diez veces seguidas el saldo de Ana. El campo leidaEn cambia en las diez respuestas: las diez salieron de Postgres. Después conté las lecturas en el log de las tres copias de cuenta y revisé los contadores de Redis.

```
oswal@OswaldoF MINGW64 ~/Onedrive/desktop/Mexibanco/mexi-banco-s11 (trabajo-s11)
$ curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":200}'
{"id":47,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","monto":200,"fecha":"2026-11
-07T05:04:00.416686758Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":638.00,"intentosDeCompen
sacion":0,"actualizada":"2026-11-07T05:04:00.580246488Z","eventoId":"31241e35-88ca-4629-80b6-1ea63c9a6199"}
oswal@OswaldoF MINGW64 ~/Onedrive/desktop/Mexibanco/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 10); do
  curl -s http://localhost:8080/api/cuentas/002180000000000001; echo
done
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:24.626953391Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:24.853813465Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:25.179094286Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:25.395118863Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:25.628442201Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:25.874632182Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:26.082571653Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:26.313491105Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:26.535168530Z"}
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:04:26.764144687Z"}

oswal@OswaldoF MINGW64 ~/Onedrive/desktop/Mexibanco/mexi-banco-s11 (trabajo-s11)
$ docker compose logs cuenta | grep -c "leida de Postgres"
12

oswal@OswaldoF MINGW64 ~/Onedrive/desktop/Mexibanco/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:0
keyspace_misses:0
```

RSULTADO. Diez consultas produjeron 12 lecturas a Postgres (las 10 consultas más 2 de la transferencia). Redis en cero: todavía nadie lo usa. Este es el antes.

2.

Usamos el patrón cache-aside: la app pregunta primero a Redis y, si no está la copia, lee Postgres y deja una copia para la siguiente consulta. Cuando el saldo cambia, la copia se borra (no se actualiza) y la siguiente consulta la vuelve a llenar. Son tres piezas.

Archivo	Qué hice y por qué
ConfiguracionCache	Agregué @EnableCaching (sin eso las anotaciones se ignoran sin avisar) y configuré la caché cuentas con tres cosas: entryTtl(30s) para que ninguna copia dure para siempre; serializador JSON de CuentaVista para poder leerla con redis-cli GET en vez de bytes binarios de Java; y transactionAware() para que el DEL espere al commit.
ConsultaDeSaldo.consultar	@Cacheable(cacheNames = CUENTAS, key = "#clabe"). La clave en Redis queda cuentas::CLABE. En un hit el método ni corre, por eso Postgres ni se entera.
CuentaService.cargar / abonar	@CacheEvict(cacheNames = CUENTAS, key = "#clabe") junto al @Transactional. Cada cargo o abono borra la copia de esa cuenta.
CuentaService.consultar	**No se toca.** Es la lectura con la que cargar decide si alcanza el saldo y siempre va a Postgres. La caché muestra, nunca decide.

Predicción: seis consultas contestadas por distintas copias de cuenta deberían leer Postgres una sola vez, porque la caché vive en Redis y no en la memoria de cada copia.

```
oswal@OswaldoF MINGW64 ~/Onedrive/desktop/Mexibanco/mexi-banco-s11 (trabajo-s11)
$ for i in 1 2 3 4 5 6; do
  curl -s -D - http://localhost:8080/api/cuentas/002180000000000001 | grep -iE 'x-instancia|leidaEn'
done; \
docker compose logs --since 30s cuenta | grep -c "leida de Postgres"; \
docker compose exec -T redis redis-cli GET cuentas::002180000000000001; \
docker compose exec -T redis redis-cli TTL cuentas::002180000000000001
X-Instancia: b22fa4f8f1d9
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:17:35.035023006Z"}
X-Instancia: c6f7401b16b7
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:17:35.035023006Z"}
X-Instancia: 45ca4834a724
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:17:35.035023006Z"}
X-Instancia: b22fa4f8f1d9
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:17:35.035023006Z"}
X-Instancia: c6f7401b16b7
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:17:35.035023006Z"}
X-Instancia: 45ca4834a724
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:17:35.035023006Z"}
1
{"clabe":"002180000000000001","titular":"Ana","saldo":638.00,"leidaEn":"2026-11-07T05:17:35.035023006Z"}
26
```

RESULTADO. La instancia cambia pero leidaEn no: las tres copias comparten una sola entrada en Redis. Una lectura a Postgres en lugar de seis, y el valor en Redis es JSON legible con TTL.

Con redis-cli MONITOR abierto en otra terminal transferí 10 de Ana a Beto. Se ven los DEL de las dos cuentas después de los commits del cargo y del abono, y luego mi consulta como miss: un GET seguido de un SET ... PX 30000 con el saldo nuevo (618.00).

```
1794029098.106304 [0 172.21.0.7:58670] "DEL" "cuentas::002180000000000001"
1794029099.584160 [0 172.21.0.14:60904] "GET" "cuentas::002180000000000001"
1794029099.588887 [0 172.21.0.14:60904] "SET" "cuentas::002180000000000001" "{\"clabe\":\"002180000000000001\",\"titular\":\"Ana\",
\"saldo\":628.00,\"leidaEn\":\"2026-11-07T05:24:59.588043879Z\"}" "PX" "30000"
1794029099.893406 [0 172.21.0.14:60904] "GET" "cuentas::002180000000000001"
1794029099.900159 [0 172.21.0.8:41552] "GET" "cuentas::002180000000000002"
1794029099.905161 [0 172.21.0.8:41552] "SET" "cuentas::002180000000000002" "{\"clabe\":\"002180000000000002\",\"titular\":\"Beto\",
\"saldo\":872.00,\"leidaEn\":\"2026-11-07T05:24:59.904126328Z\"}" "PX" "30000"
1794029099.930281 [0 172.21.0.7:58670] "DEL" "cuentas::002180000000000001"
1794029100.048034 [0 172.21.0.14:60904] "DEL" "cuentas::002180000000000002"
1794029100.882758 [0 127.0.0.1:35090] "EXISTS" "cuentas::002180000000000001"
1794029100.934815 [0 127.0.0.1:35100] "ping"
1794029101.233097 [0 172.21.0.8:41552] "GET" "cuentas::002180000000000001"
1794029101.237836 [0 172.21.0.8:41552] "SET" "cuentas::002180000000000001" "{\"clabe\":\"002180000000000001\",\"titular\":\"Ana\",\
"saldo\":618.00,\"leidaEn\":\"2026-11-07T05:25:01.236885295Z\"}" "PX" "30000"
```

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001 > /dev/null; \
   curl -s -X POST http://localhost:8080/api/transferencias -H 'Content-Type: appli
cation/json' \
     -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","m
onto":10}'; echo; \
   docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001; \
   curl -s http://localhost:8080/api/cuentas/002180000000000001
{"id":49,"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","mo
nto":10,"fecha":"2026-11-07T05:24:59.907687167Z","estado":"COMPLETADA","motivo":nul
l,"saldoOrigenTrasCargo":618.00,"intentosDeCompensacion":0,"actualizada":"2026-11-0
7T05:25:00.075865086Z","eventoId":"8a7d91b1-8fa4-4b1e-bea7-cff47e1b46dc"}
0
{"clabe":"002180000000000001","titular":"Ana","saldo":618.00,"leidaEn":"2026-11-07T
05:25:01.236885295Z"}
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli EXISTS cuentas::002180000000000001
0

oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001
{"clabe":"002180000000000001","titular":"Ana","saldo":618.00,"leidaEn":"2026-11-07T
05:29:43.347600647Z"}
```

RESULTADO. Tras la transferencia EXISTS da 0 y la siguiente consulta devuelve el saldo actualizado y un leidaEn nuevo: la copia se borró y se volvió a llenar desde Postgres.

3. Mis medidas

Hit ratio = hits / (hits + misses). Antes de cada escenario corrí CONFIG RESETSTAT para poner los contadores en cero.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT
OK

oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ for i in $(seq 1 100); do curl -s -o /dev/null http://localhost:8080/api/cuentas/
002180000000000001; done; \
docker compose exec -T redis redis-cli INFO stats | grep keyspace
keyspace_hits:97
keyspace_misses:3
```

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT; \
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
  if [ $((i % 10)) -eq 0 ]; then
    curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Conte
nt-Type: application/json' \
      -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","
monto":1}'
  fi
done; \
docker compose exec -T redis redis-cli INFO stats | grep keyspac
OK
keyspace_hits:98
keyspace_misses:22
```

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T redis redis-cli CONFIG RESETSTAT; \
for i in $(seq 1 100); do
  curl -s -o /dev/null http://localhost:8080/api/cuentas/002180000000000001
  if [ $((i % 10)) -eq 0 ]; then
    curl -s -o /dev/null -X POST http://localhost:8080/api/transferencias -H 'Conte
nt-Type: application/json' \
      -d '{"claveOrigen":"002180000000000001","claveDestino":"002180000000000002","
monto":1}'
  fi
done; \
docker compose exec -T redis redis-cli INFO stats | grep keyspac
OK
keyspace_hits:0
keyspace_misses:0
```

Escenario	Hits	Misses	Hit ratio
A: 100 consultas	97	3	97 %
B: 100 consultas + 10 transferencias (con caché)	98	22	81.7 %

Por qué baja en B. Cada transferencia borra la copia de Ana y la de Beto. La siguiente lectura de cada uno falla y vuelve a llenar la caché; con diez transferencias eso son decenas de misses extra. Los 3 misses de A seguramente son el primer llenado y alguna copia que caducó por el TTL de 30 s mientras corría el ciclo.

Los avisos en MongoDB.

Cada aviso es un documento con campos comunes (eventoId, transferenciaId, clabe, tipo, monto, saldo, fecha, canal) más los de su canal. El _id lo pone el banco como eventoId:clabe, así un evento repetido con la misma CLABE no puede guardarse dos veces. El dinero va en Decimal128, nunca en double.

Método en BitacoraDeAvisos	Qué hace
documento(...)	Arma el Document con los campos comunes y un switch sobre el Contacto que agrega los del canal: SMS (telefono, texto), CORREO (correo, asunto, cuerpo) y PUSH (token, titulo, cuerpo). Como Contacto es una interfaz sellada, un canal olvidado no compila.
registrar(evento)	Inserta un aviso por destinatario, uno por uno. Si Mongo responde DuplicateKeyException el aviso ya estaba: se registra en el log y se sigue con el siguiente. Devuelve cuántos eran nuevos.
crearIndice()	Crea al arrancar el índice clabe_fecha = { clabe: 1, fecha: -1 }, que sigue la forma de la consulta de “últimos”.
ultimos(clabe)	Query con clabe igual a la recibida, ordenada por fecha descendente y limit(10).
ConsumidorDeTransferencias	Cambié una sola línea: ahora guarda con bitacora.registrar(evento) y ya no en la bandeja de Postgres. Los avisos se mudan, no se copian a dos bases.

Hice una transferencia de 200 de Ana a Beto y otra de 50 de Ana a Carla, y consulté la colección ordenada por fecha: cuatro avisos con la forma de su canal.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-sll (trabajo-sll)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find().sort({fecha: 1})'
[
  {
    _id: '6158ce17-3544-4361-865b-e9091ee66432:002180000000000001',
    eventoId: '6158ce17-3544-4361-865b-e9091ee66432',
    transferenciaId: Long('70'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('200'),
    saldo: Decimal128('398.00'),
    fecha: ISODate('2026-11-07T06:14:10.094Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 200, saldo resultante 398.00'
  },
  {
    _id: '6158ce17-3544-4361-865b-e9091ee66432:002180000000000002',
    eventoId: '6158ce17-3544-4361-865b-e9091ee66432',
    transferenciaId: Long('70'),
    clabe: '002180000000000002',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('200'),
    saldo: Decimal128('1102.00'),
    fecha: ISODate('2026-11-07T06:14:10.094Z'),
    canal: 'CORREO',
    correo: 'beto@correo.mx',
    asunto: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 200, saldo resultante 1102.00'
  },
```

```
  {
    _id: 'edfebcb4-f903-45bd-8d0a-3335c0c3a204:002180000000000001',
    eventoId: 'edfebcb4-f903-45bd-8d0a-3335c0c3a204',
    transferenciaId: Long('71'),
    clabe: '002180000000000001',
    tipo: 'TRANSFERENCIA_ENVIADA',
    monto: Decimal128('50'),
    saldo: Decimal128('348.00'),
    fecha: ISODate('2026-11-07T06:14:56.915Z'),
    canal: 'SMS',
    telefono: '+52 55 5555 0101',
    texto: 'TRANSFERENCIA_ENVIADA por 50, saldo resultante 348.00'
  },
  {
    _id: 'edfebcb4-f903-45bd-8d0a-3335c0c3a204:002180000000000003',
    eventoId: 'edfebcb4-f903-45bd-8d0a-3335c0c3a204',
    transferenciaId: Long('71'),
    clabe: '002180000000000003',
    tipo: 'TRANSFERENCIA_RECIBIDA',
    monto: Decimal128('50'),
    saldo: Decimal128('350.00'),
    fecha: ISODate('2026-11-07T06:14:56.915Z'),
    canal: 'PUSH',
    token: 'fcm:demo-002180000000000003',
    titulo: 'Recibiste una transferencia',
    cuerpo: 'TRANSFERENCIA_RECIBIDA por 50, saldo resultante 350.00'
  }
]
```

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({telefono: {$exists: true}}, {clabe: 1,
 canal: 1, telefono: 1})'
[
  {
    _id: '6158ce17-3544-4361-865b-e9091ee66432:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  },
  {
    _id: 'edfebcb4-f903-45bd-8d0a-3335c0c3a204:002180000000000001',
    clabe: '002180000000000001',
    canal: 'SMS',
    telefono: '+52 55 5555 0101'
  }
]
```

RESULTADO. Cuatro avisos: dos SMS de Ana, un CORREO de Beto y un PUSH de Carla, cada uno con su forma. La consulta por telefono solo trae los dos de Ana.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ curl -s http://localhost:8080/api/notificaciones/002180000000000002
[{"_id":"59ce1d63-2c5d-48b6-b6b4-a17a6fabcfa3:002180000000000002","eventoId":"59ce1d63-2c5d-48b6-b6b4-a17a6fabcfa3
","transferenciaId":83,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1114.00,"fec
ha":"2026-11-07T06:18:03.865+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA
_RECIBIDA por 1, saldo resultante 1114.00"},{"_id":"8b56ee68-8a42-4a81-8451-dd168ebe2b9b:002180000000000002","eventoId":"8b56ee68-8a42-4a81-8
451-dd168ebe2b9b","transferenciaId":82,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1113.00,"fecha":"2026-1
1-07T06:18:03.544+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA p
or 1, saldo resultante 1113.00"},{"_id":"2d9fcb99-bcfa-429c-af16-1112920eaf4c:002180000000000002","eventoId":"2d9fcb99-bcfa-429c-af16-1112920
eaf4c","transferenciaId":81,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1112.00,"fecha":"2026-11-07T06:18:
03.177+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo
 resultante 1112.00"},{"_id":"db6465b1-54e6-4974-9b7c-16ad91b71f1e:002180000000000002","eventoId":"db6465b1-54e6-4974-9b7c-16ad91b71f1e","tra
nsferenciaId":80,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1111.00,"fecha":"2026-11-07T06:18:02.921+00:0
0","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante
 1111.00"},{"_id":"50e399c5-4329-4ae0-abf8-23f89d3da31e:002180000000000002","eventoId":"50e399c5-4329-4ae0-abf8-23f89d3da31e","transferenciaI
d":79,"clabe":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1110.00,"fecha":"2026-11-07T06:18:02.661+00:00","canal":
"CORREO","correo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1110.00"},
{"_id":"137960c7-05c4-49ba-8c14-b5c562c7c887:002180000000000002","eventoId":"137960c7-05c4-49ba-8c14-b5c562c7c887","transferenciaId":78,"clab
e":"002180000000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1109.00,"fecha":"2026-11-07T06:18:02.414+00:00","canal":"CORREO","c
orreo":"beto@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1109.00"},{"_id":"15b
b6380-b32d-480c-8c5a-3932fdb28704:002180000000000002","eventoId":"15bb6380-b32d-480c-8c5a-3932fdb28704","transferenciaId":77,"clabe":"0021800
00000000002","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1108.00,"fecha":"2026-11-07T06:18:02.115+00:00","canal":"CORREO","correo":"bet
o@correo.mx","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1108.00"},{"_id":"6f268116-eb13-
47b3-bf1d-e854e15d727f:002180000000000002","eventoId":"6f268116-eb13-47b3-bf1d-e854e15d727f","transferenciaId":76,"clabe":"002180000000000002
","tipo":"TRANSFERENCIA_RECIBIDA","monto":1,"saldo":1107.00,"fecha":"2026-11-07T06:18:01.793+00:00","canal":"CORREO","correo":"beto@correo.mx
","asunto":"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1107.00"},{"_id":"7bc1fdd5-ad2c-4309-9877-d
556dd619850:002180000000000002","eventoId":"7bc1fdd5-ad2c-4309-9877-d556dd619850","transferenciaId":75,"clabe":"002180000000000002","tipo":"T
RANSFERENCIA_RECIBIDA","monto":1,"saldo":1106.00,"fecha":"2026-11-07T06:18:01.524+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":
"Recibiste una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1106.00"},{"_id":"9edddd89-6d49-4321-b933-0537212b044f
:002180000000000002","eventoId":"9edddd89-6d49-4321-b933-0537212b044f","transferenciaId":74,"clabe":"002180000000000002","tipo":"TRANSFERENCI
A_RECIBIDA","monto":1,"saldo":1105.00,"fecha":"2026-11-07T06:18:01.223+00:00","canal":"CORREO","correo":"beto@correo.mx","asunto":"Recibiste
una transferencia","cuerpo":"TRANSFERENCIA_RECIBIDA por 1, saldo resultante 1105.00"}]
```

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.getIndexes()'
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { clabe: 1, fecha: -1 }, name: 'clabe_fecha' }
]

oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.find({clabe: "002180000000000002"}).sort({fecha: -1}).limit(1
0).explain("executionStats")'
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'notificacion.avisos',
    parsedQuery: { clabe: { '$eq': '002180000000000002' } },
    indexFilterSet: false,
    queryHash: 'C73271DB',
    planCacheShapeHash: 'C73271DB',
    planCacheKey: '501A071D',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 10,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { clabe: 1, fecha: -1 },
          indexName: 'clabe_fecha',
          isMultiKey: false,
          multiKeyPaths: { clabe: [], fecha: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
```

Publiqué el mismo evento (eventoId 11111111-2222-4333-8444-555555555555) dos veces en el exchange transferencias con CLABEs inventadas (...091 y ...092). Predicción: cada evento genera dos avisos, así que aun con dos eventos iguales deben quedar 2 documentos.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s11 (trabajo-s11)
$ for i in 1 2; do
curl -u mexibanco:mexibanco -X POST http://localhost:15672/api/exchanges/%2F/transferencias/publish \
  -H 'Content-Type: application/json' \
  -d '{"properties":{"delivery_mode":2,"content_type":"application/json"},"routing_key":"transferencia.realizada","payload_encoding
":"string","payload":"{\"eventoId\":\"11111111-2222-4333-8444-555555555555\",\"tipo\":\"TransferenciaRealizada\",\"transferenciaId\
":9001,\"clabeOrigen\":\"002180000000000091\",\"clabeDestino\":\"002180000000000092\",\"monto\":25,\"saldoOrigen\":475,\"saldoDesti
no\":525,\"fecha\":\"2026-10-31T13:00:00Z\"}"}'; echo
done
sleep 5
docker compose exec -T mongo mongosh notificacion --eval 'db.avisos.countDocuments({eventoId: "11111111-2222-4333-8444-555555555555
"})'
docker compose logs --since 2m notificacion | grep "ya estaba"
{"routed":true}
{"routed":true}
2
notificacion-1  | 2026-11-07T06:26:22.070Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos           :
[avisos] 11111111-2222-4333-8444-555555555555:002180000000000091 ya estaba: evento repetido, no se duplica
notificacion-1  | 2026-11-07T06:26:22.080Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos           :
[avisos] 11111111-2222-4333-8444-555555555555:002180000000000092 ya estaba: evento repetido, no se duplica
notificacion-1  | 2026-11-07T06:26:22.304Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos           :
[avisos] 11111111-2222-4333-8444-555555555555:002180000000000091 ya estaba: evento repetido, no se duplica
notificacion-1  | 2026-11-07T06:26:22.308Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.avisos.BitacoraDeAvisos           :
[avisos] 11111111-2222-4333-8444-555555555555:002180000000000092 ya estaba: evento repetido, no se duplica
```

RESULTADO. El mismo evento dos veces dejó dos avisos, no cuatro. El _id = eventoId:clabe hace que Mongo rechace el segundo insert y registrar lo trata como “ya estaba”, no como error. Así la entrega “al menos una vez” de RabbitMQ no duplica avisos.

```
oswal@OswaldoF MINGW64 ~/onedrive/desktop/mexibanco/mexi-banco-s
11 (trabajo-s11)
$ curl -s http://localhost:8080/api/cuentas/002180000000000001;
echo
docker compose exec -T db psql -U mexibanco -d cuenta -c "update
 cuenta set saldo = 5 where clabe = '002180000000000001'"
curl -s -X POST http://localhost:8080/api/transferencias -H 'Con
tent-Type: application/json' \
  -d '{"claveOrigen":"002180000000000001","claveDestino":"002180
000000000002","monto":100}'; echo
for i in $(seq 1 8); do
  docker compose exec -T redis redis-cli TTL cuentas::0021800000
00000001
  curl -s http://localhost:8080/api/cuentas/002180000000000001;
echo
  sleep 5
done
{"clabe":"002180000000000001","titular":"Ana","saldo":518.00,"le
idaEn":"2026-11-07T06:49:06.778479245Z"}
UPDATE 1
{"type":"about:blank","title":"Unprocessable Entity","status":42
2,"detail":"Saldo insuficiente en la CLABE 002180000000000001","
instance":"/transferencias","servicio":"cuenta"}
-2
{"clabe":"002180000000000001","titular":"Ana","saldo":5.00,"leid
aEn":"2026-11-07T06:49:10.512622413Z"}
24
```

¿Pasó la transferencia de 100? No: respondió 422 Unprocessable Entity, “Saldo insuficiente”. Aunque el cliente podía seguir viendo 518 en la app, el cargo lee el saldo de Postgres, que ya valía 5. Esa es justo la razón de que CuentaService.consultar no esté cacheada.

Preguntas de cierre.

1. ¿Por qué MongoDB para los avisos y no para el saldo?

Los avisos tienen una forma distinta por canal (SMS lleva teléfono y texto, correo lleva dirección, asunto y cuerpo, push lleva token), y en una tabla serían muchas columnas casi vacías. Además se escriben una vez, nunca se actualizan y se leen enteros por CLABE y fecha, justo lo que resuelve un documento con un índice clabe_fecha. El saldo es dinero: necesita transacciones, bloqueo optimista (@Version) y que nunca se contradiga ni se pierda. Eso es la verdad del banco y se queda en Postgres.

2. ¿Qué hit ratio medí y qué compré con la caché?

Medí 97 % en A (97 hits, 3 misses) y 81.7 % en B (98 hits, 22 misses). Baja en B porque cada transferencia borra las copias de Ana y de Beto y la siguiente lectura de cada uno es un miss. Con la caché, las cien consultas de A casi no tocan Postgres; la ganancia principal es capacidad y, en segundo lugar, velocidad.

3. ¿Qué pasaría si la autorización del cargo leyera el saldo de la caché?

Autorizaría con un saldo que puede tener hasta 30 s de atraso. En la Parte 4 la caché seguía mostrando 518 cuando el saldo real era 5; si el cargo hubiera usado esa copia, la transferencia de 100 habría pasado y la cuenta quedaría en negativo. Por eso el cargo lee Postgres y respondió 422.

4. ¿Por qué DEL y no SET con el saldo nuevo? ¿Y por qué después del commit?

DEL en vez de SET: la caché es una copia que se puede reconstruir, y la verdad está en Postgres. Borrar es simple y seguro: la siguiente consulta la vuelve a llenar con el saldo real. Un SET obliga a calcular el valor correcto en el código y puede dejar un dato equivocado si dos operaciones concurrentes se cruzan o si la transacción termina en rollback. Después del commit: si se borra antes, otra consulta puede leer de Postgres el saldo viejo (el cambio todavía no está confirmado) y volver a guardarlo en la caché, y la copia vieja quedaría hasta que expire el TTL. Con transactionAware() el DEL espera al commit y evita esa ventana.